

Constraints: Swinging Things

Hand the final code *rigidbody2d_cc.py* in on avenue that contains all test cases that can be run by changing the scenario variable. For the written answers and discussion, hand-in a separate pdf file with your answers.

Start with rigidbody2d_cc0.py main code available on avenue. Download constraint.zip from code/constraints in the content section. It contains all the required files:

Cube.egg

Tube.egg

rigidbody2d_cc0.py

rigidbody2d_mod.py

rigidbody2d_markers_mod.py

rigidbody2d_collide_mod.py

rigidbody2d_constraints_mod.py

This code combines a collision solver with a constraint solver for hinges. As described in the VG3_Constraints.pdf lecture notes, it has a generic set-up for creating hinge constraints. There is demo code there for two connected boxes (scenario 0) and a rolling car (scenario 1). For your work, rename the main file to: *rigidbody2d_cc0.py* to *rigidbody2d_cc.py* You should not need to modify other files.

Exercise 1: Set up a block (square) and a pendulum (cylinder) as scenario 2.

Model the block any way you want. It just needs to have inverse mass and inverse moment of inertia of zero. Another key thing is that its lower edge must be around $z=2$ so you can set-up the pendulum hinge to be at $x=0$, $y=3$, $z=2$. Note also that all objects in this scene need to have the same y coordinate (e.g. $y=3$).

Model the pendulum bob with a cylinder of mass, $m=1.0$ and radius $r=0.25$. The bob centre of mass should be at $x=0$, $y=3$, $z=0$. The constraint should be a long lever arm from the bob centre all the way to $x=0$, $y=3$, $z=2$ (length 2). This is outside the bob radius but that is not an issue, it will work fine. This can be modeled as a hinge.

Try it out by setting an initial speed to the left, e.g. $v = (-2, 0, 0)$.

Exercise 2: Add code to measure the period of the pendulum. You can look at `self.t` for the time and synchronize a new period when the sign of the bob's x changes (e.g. $x * x_{old} \leq 0$). This happens twice per period. Compare this to the expectation for an ideal pendulum: Period $T = 2 \pi \sqrt{L/g}$.

Careful estimating the period. Report your estimate here. Try large variations in the initial x velocity. Does it change the answer? The ideal pendulum is actually in the limit of small velocities (<1). Confirm this.

Exercise 3: Newton's Cradle. Set up scenario 3 as Newton's cradle. Your code should have a variable set at the start called `self.nSphere` to select how many spheres are involved. Otherwise it is quite similar to a pendulum, except `self.nSphere` of them. Used `nSphere=3` as a default. In the classic set-up, only one of the end spheres is moving initially and every time it collides, the sphere on the other end should move. Ideally, only one sphere (on an end) is moving at any given time.

For this exercise, we start use the standard velocity bias on collisions suggested by Erin Catto:

```
vbias = 0 if (dclose<0.01) else dclose*0.3/dt
```

Test your code with nSphere =3 and a moderate initial velocity, e.g. $v=(-2,0,0)$, for the leftmost sphere. Describe what happens. It should also work for nSphere > 3.

Exercise 4: Velocity bias testing. The default velocity bias may not be ideal for this test.

Try with vbias = 0 and report what happens.

Exercise 5: Alternative ideas for bias.

Suggest an alternative bias that works for the Newton's cradle case *and* for stacking (continuous contacts). If possible, try to think of something general that can differentiate between static and dynamic collisions. For example, just reducing the coefficient (from 0.3) is likely to only work for a specific scenario. Note that the constraints also have a bias factor to consider related to the vector offset of the constraint points.

Test your ideas. Also test it on the stacking case for HW6 (e.g. Include the stacking case from HW6 as scenario 4). Ideally your idea works for scenario 3 and 4. Experiment and hand in your best idea. Comment on your idea and its behaviour in your hand-in.

Hand in your code with scenario=3 set up to run (rigidbody2d_cc.py)